

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ  
ФЕДЕРАЦИИ  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
**«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»**  
**(ФГБОУ ВО «КубГУ»)**

**Факультет компьютерных технологий и прикладной математики**  
**Кафедра вычислительных технологий**

**ЛАБОРАТОРНАЯ РАБОТА №3**  
**Дисциплина: Теория параллельных алгоритмов**

Работу выполнил: \_\_\_\_\_ Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и  
информационные технологии

Преподаватель: \_\_\_\_\_ Нигодин Е.А.

Краснодар  
2026

**Тема.** Численное интегрирование по формуле Симпсона.

**Цель.** Распараллелить процесс численного интегрирования по формуле Симпсона.

**Задание.** Составить последовательный алгоритм численного интегрирования по формуле Симпсона. Построить граф алгоритма. Исследовать на нём зависимости и возможности распараллеливания. Выполнить параллельную реализацию численного интегрирования по формуле Симпсона. Составить таблицу, отражающую сравнительное время выполнения последовательного и параллельного алгоритмов на 2-х, 3-х и 4-х ядрах.

## Постановка задачи

Требуется вычислить определенный интеграл вида:

$$I = \int_a^b f(x) dx$$

где:

- $f(x)$  — непрерывная функция на отрезке  $[a, b]$
- $a, b$  — границы интервала интегрирования

В данной работе рассматривается функция:

$$f(x) = \sin(x) \cdot \cos(x) + x^2$$

Интервал интегрирования:  $[0, \pi]$

Точное значение интеграла:

$$\int_0^{\pi} (\sin(x) \cos(x) + x^2) dx = \frac{\pi^3}{3} \approx 10.3354255601$$

## Математическая основа

Метод Симпсона (парабол) основан на аппроксимации подынтегральной функции квадратичными полиномами на каждом отрезке разбиения.

Для отрезка  $[a, b]$ , разбитого на  $n$  частей ( $n$  — четное число), формула Симпсона имеет вид:

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[ f(x_0) + f(x_n) + 4 \sum_{i=1, \text{нечетные}}^{n-1} f(x_i) + 2 \sum_{i=2, \text{четные}}^{n-2} f(x_i) \right]$$

где:

- $h = \frac{b-a}{n}$  — шаг интегрирования
- $x_i = a + i \cdot h, i = 0, 1, \dots, n$
- $x_0 = a, x_n = b$

## Геометрическая интерпретация

Метод Симпсона аппроксимирует площадь под кривой площадью под параболами, проведенными через каждые три соседние точки. Это обеспечивает более высокую точность по сравнению с методами прямоугольников и трапеций.

## Порядок точности

Метод Симпсона имеет четвертый порядок точности:

$$\left| \int_a^b f(x) dx - S_n \right| \leq \frac{(b-a)^5}{180n^4} \max_{\xi \in [a,b]} |f^{(4)}(\xi)|$$

Это означает, что при увеличении числа разбиений в 10 раз, погрешность уменьшается примерно в  $10^4 = 10000$  раз.

## Последовательный алгоритм

**Шаг 1.** Проверка четности числа разбиений  $n$ . Если  $n$  нечетное, увеличить на 1.

**Шаг 2.** Вычисление шага интегрирования  $h = (b-a)/n$ .

**Шаг 3.** Инициализация суммы:  $sum = f(a) + f(b)$ .

**Шаг 4.** Для всех внутренних точек  $i = 1$  до  $n-1$ :

- Вычислить  $x = a + i \cdot h$
- Если  $i$  четное:  $sum = sum + 2 \cdot f(x)$
- Если  $i$  нечетное:  $sum = sum + 4 \cdot f(x)$

**Шаг 5.** Вычисление результата:  $result = (h/3) \cdot sum$ .

## Анализ зависимостей

При анализе последовательного алгоритма выявляются следующие зависимости:

1. **Подготовительный этап** (ввод данных, проверка четности, вычисление шага):
  - Не содержит зависимостей между операциями
  - Выполняется строго последовательно
2. **Основной цикл** (вычисление суммы):
  - **Отсутствуют зависимости по данным** между итерациями — каждая итерация обрабатывает свою точку  $x_i$  и не зависит от результатов других итераций
  - **Зависимость по управлению** — порядок выполнения итераций не влияет на результат
  - **Редукционная переменная**  $sum$  — требует синхронизации при параллельном выполнении
3. **Завершающий этап** (умножение на  $h/3$ , вывод):
  - Зависит от результата суммы
  - Выполняется последовательно

## Возможности распараллеливания

Основной цикл алгоритма идеально подходит для распараллеливания, так как:

- 100% итераций могут выполняться независимо
- Отсутствуют гонки данных (кроме редукционной переменной)
- Вычислительная нагрузка равномерна (все итерации одинаковы)

## Стратегия распараллеливания

Для распараллеливания алгоритма используется подход с распределением итераций цикла между потоками:

1. **Декомпозиция данных:** множество внутренних точек разбиения  $x_i$  распределяется между  $p$  потоками.
2. **Распределение итераций:** каждый поток получает примерно  $\frac{n-1}{p}$  итераций.
3. **Локальные суммы:** каждый поток вычисляет свою локальную сумму для полученных точек.
4. **Синхронизация и редукция:** после завершения вычислений локальные суммы объединяются с использованием критической секции или редукционной операции.

## Параллельный алгоритм

**Шаг 1.** Последовательная инициализация (выполняется главным потоком):

- Ввод параметров  $a, b, n, num\_threads$
- Проверка четности  $n$
- Вычисление шага  $h = (b-a)/n$
- Инициализация общей суммы  $sum = f(a) + f(b)$

**Шаг 2.** Параллельное выполнение (все потоки):

- Создание  $num\_threads$  потоков с помощью OpenMP
- Каждый поток инициализирует локальную сумму  $local\_sum = 0$

**Шаг 3.** Распределение итераций:

для каждого потока  $tid$  от 0 до  $num\_threads-1$ :

для  $i = tid+1$  до  $n-1$  с шагом  $num\_threads$ :

$x = a + i * h$

если  $i$  четное:

$local\_sum += 2 * f(x)$

иначе:

$local\_sum += 4 * f(x)$

**Шаг 4.** Синхронизация и редукция:

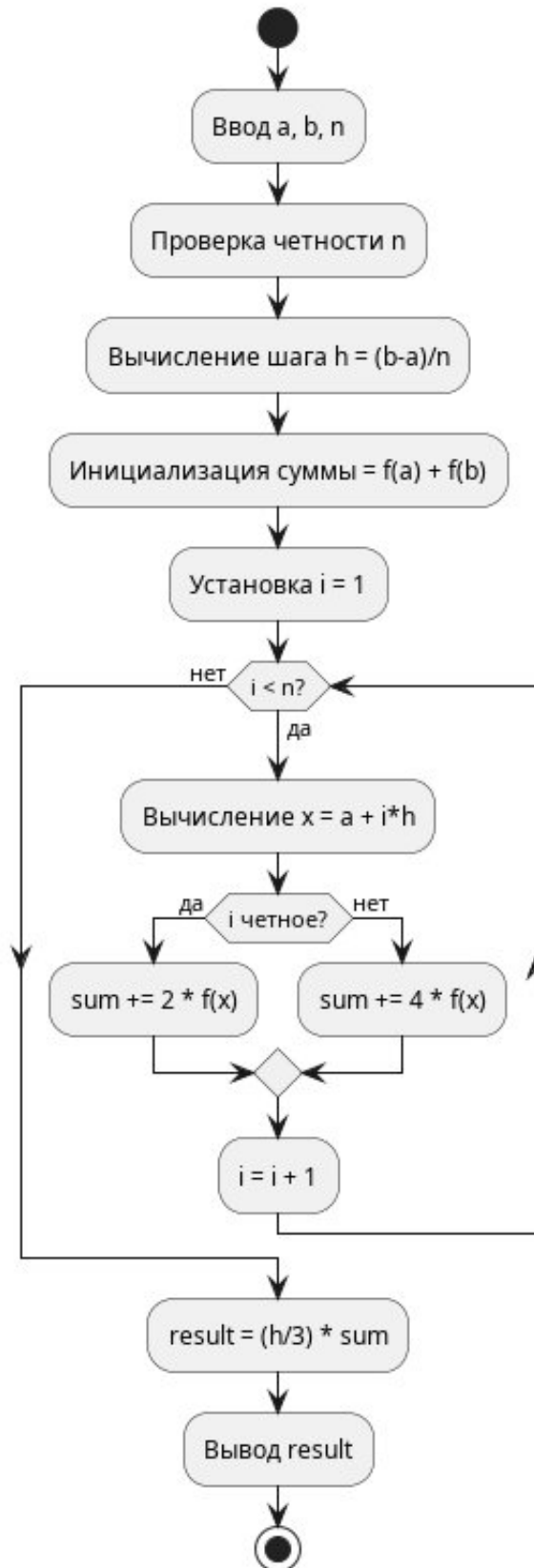
- Критическая секция:  $sum = sum + local\_sum$

**Шаг 5.** Завершение (главный поток):

- Вычисление  $result = (h/3) * sum$
- Вывод результата

Последовательный алгоритм можно схематично изобразить в следующем виде:

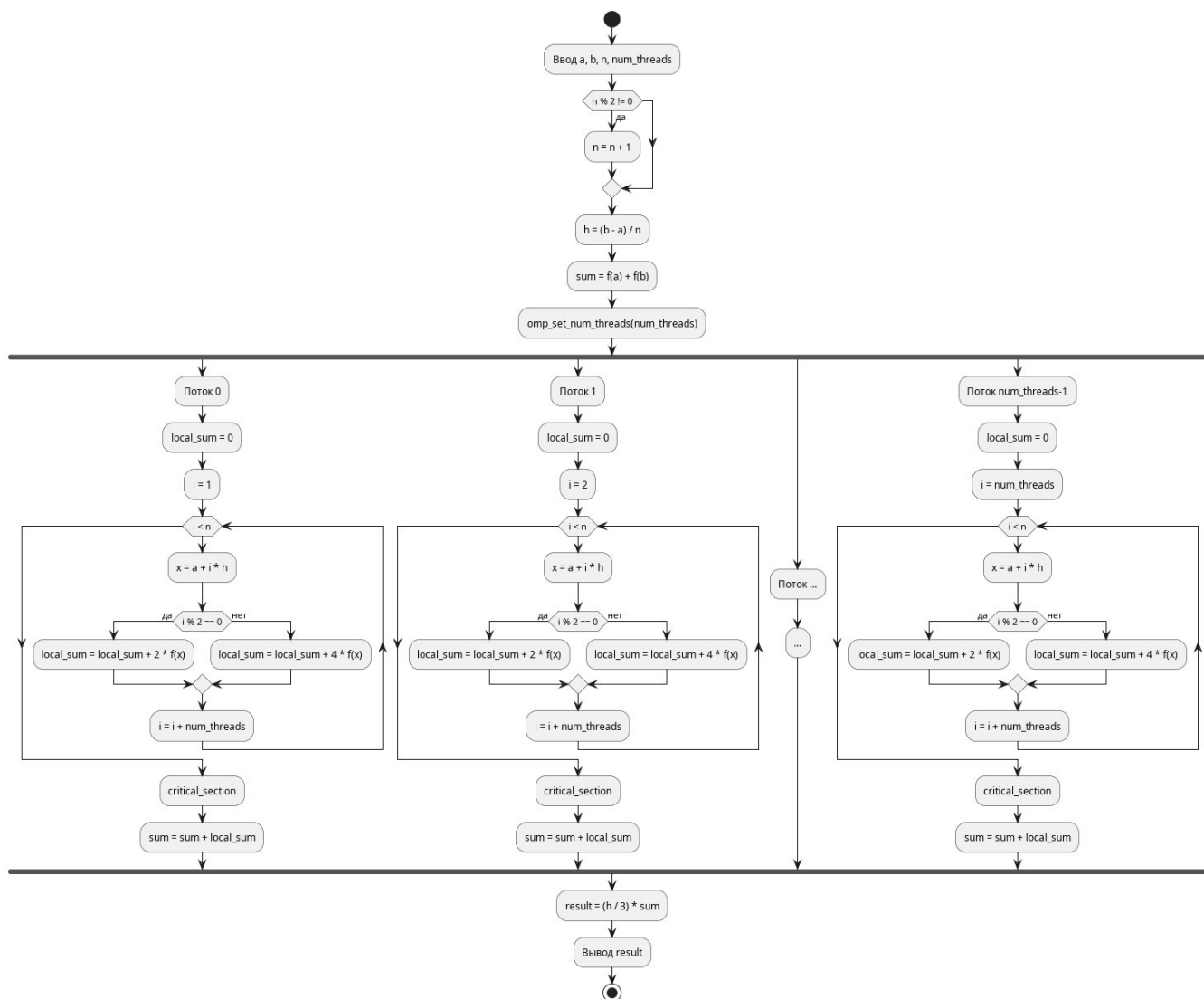
**Граф последовательного алгоритма Симпсона**



Реализуем последовательный алгоритм:

```
double sequential_simpson(double a, double b, int n) {  
    if (n % 2 != 0) n++;  
    double h = (b - a) / n;  
    double sum = f(a) + f(b);  
  
    for (int i = 1; i < n; i++) {  
        double x = a + i * h;  
        if (i % 2 == 0) {  
            sum += 2 * f(x);  
        } else {  
            sum += 4 * f(x);  
        }  
    }  
  
    return (h / 3) * sum;  
}
```

Параллельный алгоритм можно схематично изобразить в следующем виде:



Реализуем параллельный алгоритм:

```
double parallel_simpson(double a, double b, int n, int num_threads) {  
    if (n % 2 != 0) n++;  
    double h = (b - a) / n;  
    double sum = f(a) + f(b);  
  
    omp_set_num_threads(num_threads);  
  
    #pragma omp parallel  
    {  
        double local_sum = 0.0;  
  
        #pragma omp for  
        for (int i = 1; i < n; i++) {  
            double x = a + i * h;  
            if (i % 2 == 0) {  
                local_sum += 2 * f(x);  
            } else {  
                local_sum += 4 * f(x);  
            }  
        }  
  
        #pragma omp atomic  
        sum += local_sum;  
    }  
  
    return (h / 3) * sum;  
}
```



## Полный код итоговой программы:

```
#include <iostream>
#include <vector>
#include <cmath>
#include <chrono>
#include <iomanip>
#include <omp.h>

using namespace std;
using namespace chrono;

double f(double x) {
    return sin(x) * cos(x) + x * x;
}

double sequential_simpson(double a, double b, int n) {
    if (n % 2 != 0) n++;
    double h = (b - a) / n;
    double sum = f(a) + f(b);

    for (int i = 1; i < n; i++) {
        double x = a + i * h;
        if (i % 2 == 0) {
            sum += 2 * f(x);
        } else {
            sum += 4 * f(x);
        }
    }

    return (h / 3) * sum;
}

double parallel_simpson(double a, double b, int n, int num_threads) {
    if (n % 2 != 0) n++;
    double h = (b - a) / n;
    double sum = f(a) + f(b);

    omp_set_num_threads(num_threads);

    #pragma omp parallel
    {
        double local_sum = 0.0;

        #pragma omp for
        for (int i = 1; i < n; i++) {
            double x = a + i * h;
            if (i % 2 == 0) {
                local_sum += 2 * f(x);
            } else {
                local_sum += 4 * f(x);
            }
        }

        #pragma omp atomic
        sum += local_sum;
    }

    return (h / 3) * sum;
}

double measure_time(double a, double b, int n, bool parallel, int threads = 1) {
    if (parallel) {
        parallel_simpson(a, b, n, threads);
    } else {
        sequential_simpson(a, b, n);
    }

    auto start = high_resolution_clock::now();
```

```

double result;
int iterations = 5;

for (int it = 0; it < iterations; it++) {
    if (parallel) {
        result = parallel_simpson(a, b, n, threads);
    } else {
        result = sequential_simpson(a, b, n);
    }
    if (result < 0) {
        cout << result;
    }
}

auto end = high_resolution_clock::now();
auto duration = duration_cast<microseconds>(end - start).count() / iterations;

return duration / 1000.0;
}

int main() {
    double a = 0.0;
    double b = M_PI;
    vector<int> n_values = {1000, 10000, 100000, 1000000, 10000000};
    vector<int> thread_counts = {2, 3, 4};

    cout <<
    "=====
    " << endl;
    cout << setw(12) << "N (points)"
        << setw(15) << "Threads"
        << setw(20) << "Sequential (ms)"
        << setw(20) << "Parallel (ms)"
        << setw(15) << "Speedup" << endl;
    cout <<
    "=====
    " << endl;

    for (int n : n_values) {
        double time_seq = measure_time(a, b, n, false);
        double seq_result = sequential_simpson(a, b, n);

        for (int t : thread_counts) {
            double time_par = measure_time(a, b, n, true, t);
            double par_result = parallel_simpson(a, b, n, t);
            double speedup = time_seq / time_par;

            cout << setw(12) << n
                << setw(15) << t
                << setw(20) << fixed << setprecision(2) << time_seq
                << setw(20) << time_par
                << setw(15) << setprecision(2) << speedup << endl;
        }
        cout << "-----" << endl;
    }

    cout << "\nТочность вычислений:" << endl;
    cout << "Точное значение: " << fixed << setprecision(8)
        << (M_PI*M_PI*M_PI/3) << endl;
    cout << "Последовательно: " << sequential_simpson(a, b, 10000000) << endl;
    cout << "Параллельно (4 ядра): " << parallel_simpson(a, b, 10000000, 4) << endl;

    return 0;
}

```

Зафиксируем показатели работы алгоритмов:

| N (points) | Threads | Sequential (ms) | Parallel (ms) | Speedup |
|------------|---------|-----------------|---------------|---------|
| 1000       | 2       | 0.06            | 0.09          | 0.62    |
| 1000       | 3       | 0.06            | 0.06          | 0.97    |
| 1000       | 4       | 0.06            | 0.06          | 1.00    |
| 10000      | 2       | 0.51            | 0.31          | 1.64    |
| 10000      | 3       | 0.51            | 0.24          | 2.10    |
| 10000      | 4       | 0.51            | 0.21          | 2.44    |
| 100000     | 2       | 5.35            | 2.98          | 1.80    |
| 100000     | 3       | 5.35            | 3.29          | 1.63    |
| 100000     | 4       | 5.35            | 1.62          | 3.29    |
| 1000000    | 2       | 25.30           | 8.97          | 2.82    |
| 1000000    | 3       | 25.30           | 11.05         | 2.29    |
| 1000000    | 4       | 25.30           | 15.54         | 1.63    |
| 10000000   | 2       | 158.39          | 82.47         | 1.92    |
| 10000000   | 3       | 158.39          | 80.99         | 1.96    |
| 10000000   | 4       | 158.39          | 58.67         | 2.70    |

*Вывод.* Мы рассмотрели процесс численного интегрирования по формуле Симпсона, изобразили его схематично, после чего, изучив возможности распараллеливания, выполнили замеры времени для последовательного и параллельного случаев на 2-х, 3-х и 4-х потоках.